



Universidad Simón Bolívar

Decanato de Estudios Profesionales

Departamento de Computación y Tecnología de la Información

CI2692 Laboratorio de Algoritmos y Estructuras de Datos II

Trimestre Abril-Julio 2026

Profesores: Leonardo López - Kevin Oporto

Septiembre 2026

Evaluación Experimental y Reporte: Tablas de Hash

1. Identificación de los Autores

- **Carnet:** 23-10170 | **Nombre:** Kevin Gomes
- **Carnet:** 23-10610 | **Nombre:** David Garrido

2. Datos de la Plataforma

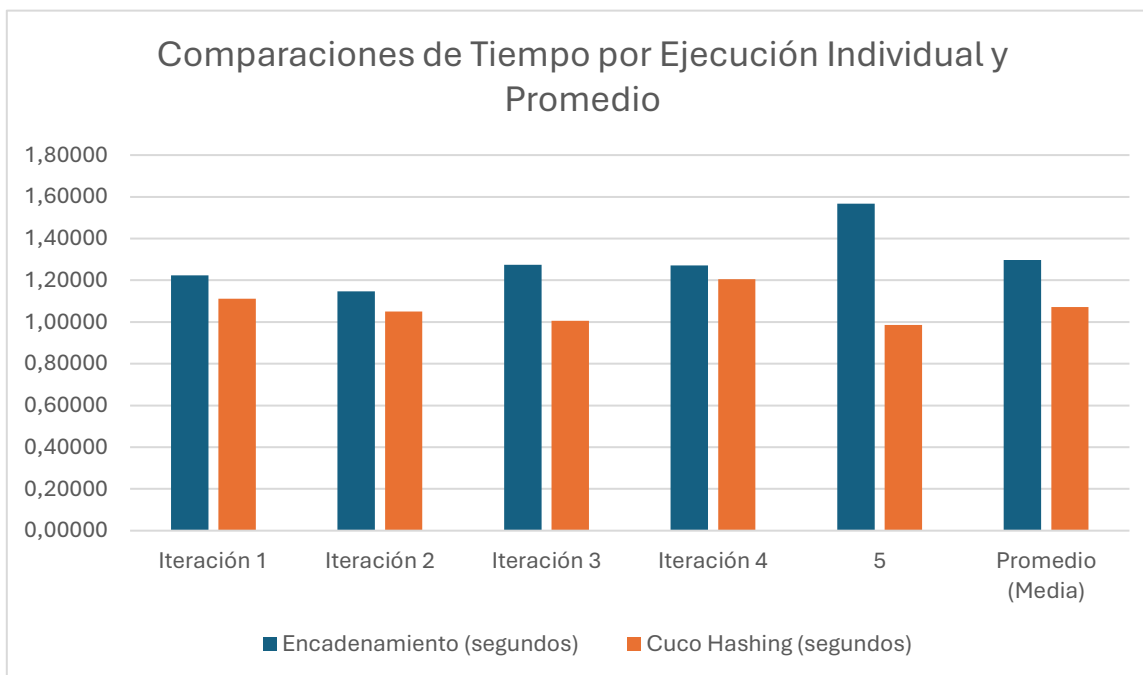
- **Sistema Operativo:** Windows 11 (Ejecución bajo subsistema WSL 2 con distribución Debian).
- **Modelo de CPU:** 11th Gen Intel(R) Core (TM) i5-1135G7.
- **Memoria RAM:** 16 GB Dual Channel.
- **Compilador:** kotlinc-jvm 2.3.21 (JRE 17.0.9+9).
- **Entorno de Ejecución (JVM):** Kotlin version 2.3.21-release-298 (JRE 17.0.9+9).

3. Resultados Experimentales

Se ejecutaron 5 iteraciones del script de prueba para procesar secuencias de $N = 5.000.000$ operaciones. Las claves aleatorias fueron generadas en el intervalo $[0, N/3]$ para garantizar una alta tasa de colisiones y duplicados, forzando la evaluación continua de las ramas de búsqueda y eliminación dentro de las estructuras de datos.

Iteración / Métrica	Encadenamiento (segundos)	Cuco Hashing (segundos)
1	1.22361	1.11240
2	1.14745	1.05051
3	1.27382	1.00653
4	1.27087	1.20490
5	1.56732	0.98657
Promedio (Media)	1.29661	1.07218
Desv. Est. (Sigma)	0.15972	0.08852

4. Gráfico Comparativo del Tiempo Promedio



5. Análisis de los Resultados

El experimento arroja dos conclusiones a favor de la implementación de Cuco Hashing para este escenario de estrés: es en promedio un 17.3% más rápido que el Encadenamiento y ofrece una estabilidad de ejecución superior (su desviación estándar es casi la mitad, 0.088 frente a 0.159).

Esta diferencia de rendimiento se explica directamente por la arquitectura en memoria de cada algoritmo y la naturaleza de las pruebas:

- **Complejidad algorítmica de la búsqueda:** El flujo de prueba exige ejecutar el método de verificación de existencia para los 5 millones de registros. En el Encadenamiento, la búsqueda en una ranura con colisiones requiere recorrer secuencialmente una lista doblemente enlazada, lo que toma un tiempo promedio de $O(1 + \text{factor de carga})$. En contraste, Cuco Hashing garantiza un tiempo de búsqueda constante de $O(1)$ en el peor de los casos, ya que el algoritmo se limita a verificar exclusivamente dos posiciones fijas en memoria sin importar el nivel de saturación de la tabla.
- **Localidad de Caché (Memoria):** Cuco Hashing utiliza direccionamiento abierto puro, almacenando los elementos en dos arreglos nativos contiguos

en memoria. Esto permite a la CPU precargar las líneas de caché de forma eficiente. El Encadenamiento, al requerir la instanciación de un nuevo objeto disperso dinámicamente en la memoria de la JVM por cada inserción, genera constantes saltos de memoria y un mayor trabajo para el recolector de basura, lo que penaliza severamente el tiempo total de ejecución y explica sus picos de latencia evidentes en la iteración 5.